ECE3411

Lecture 11

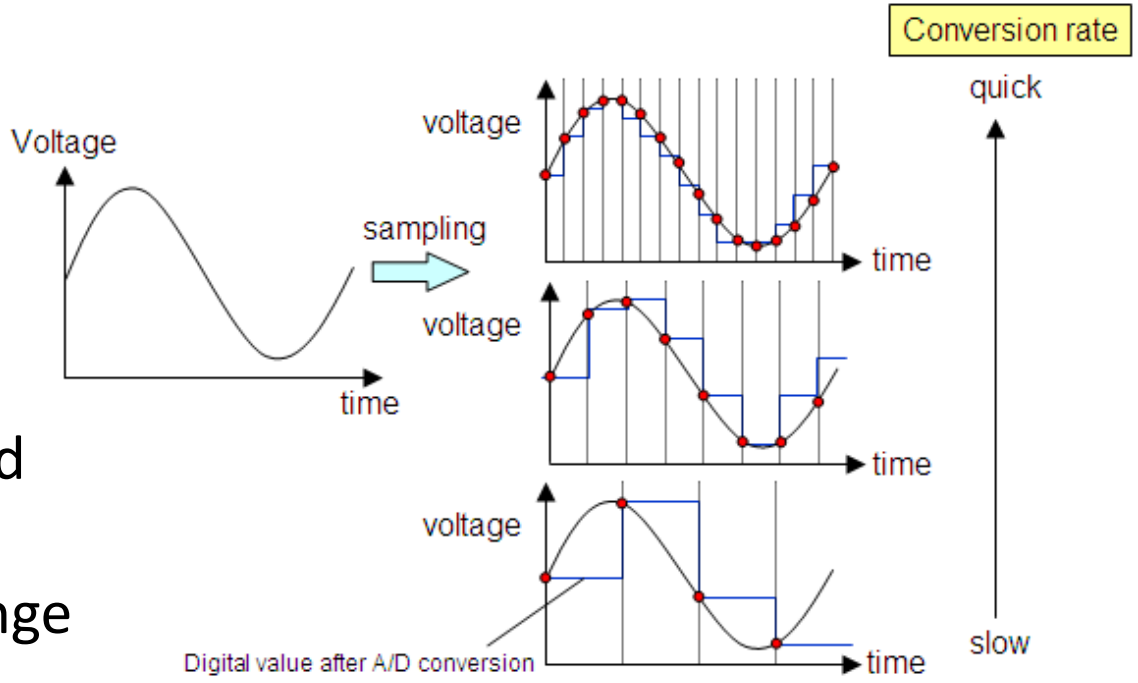
Analog Digital Converter

Sung Yeul Park
Department of Electrical & Computer Engineering
University of Connecticut
Email: sung_yeul.park@uconn.edu

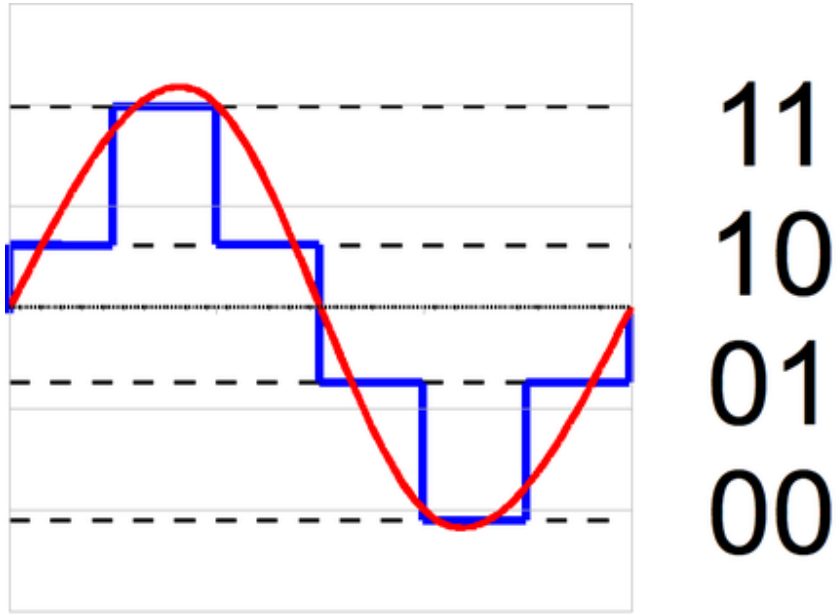
Adapted from John Chandy ECE3411 – Fall 2024

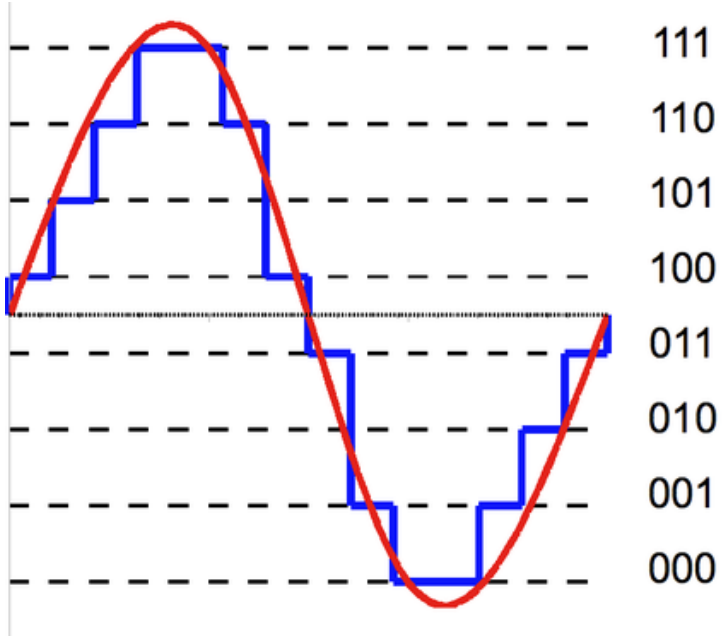
Introduction

- Why do we need Analog-Digital Conversion?
  - Real world is Analog
  - Digital computers process Digital signals
  - ADC/DAC serves as the interface between computers and the real world
- Analog Signals are “Continuous”
  - A “Discrete” version of the analog signal is created by “Sampling” the analog signal
  - ADC then maps each sample onto a quantized range of voltages which can be represented by binary values.



Introduction



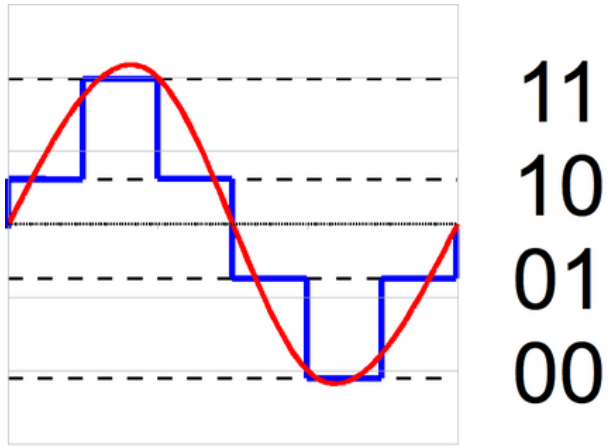


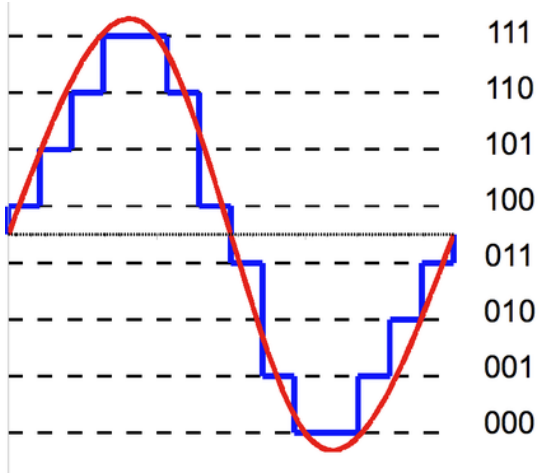
Which signals need to be converted?

From Wikipedia: https://en.wikipedia.org/wiki/Quantization_(signal_processing)

ADC Terms

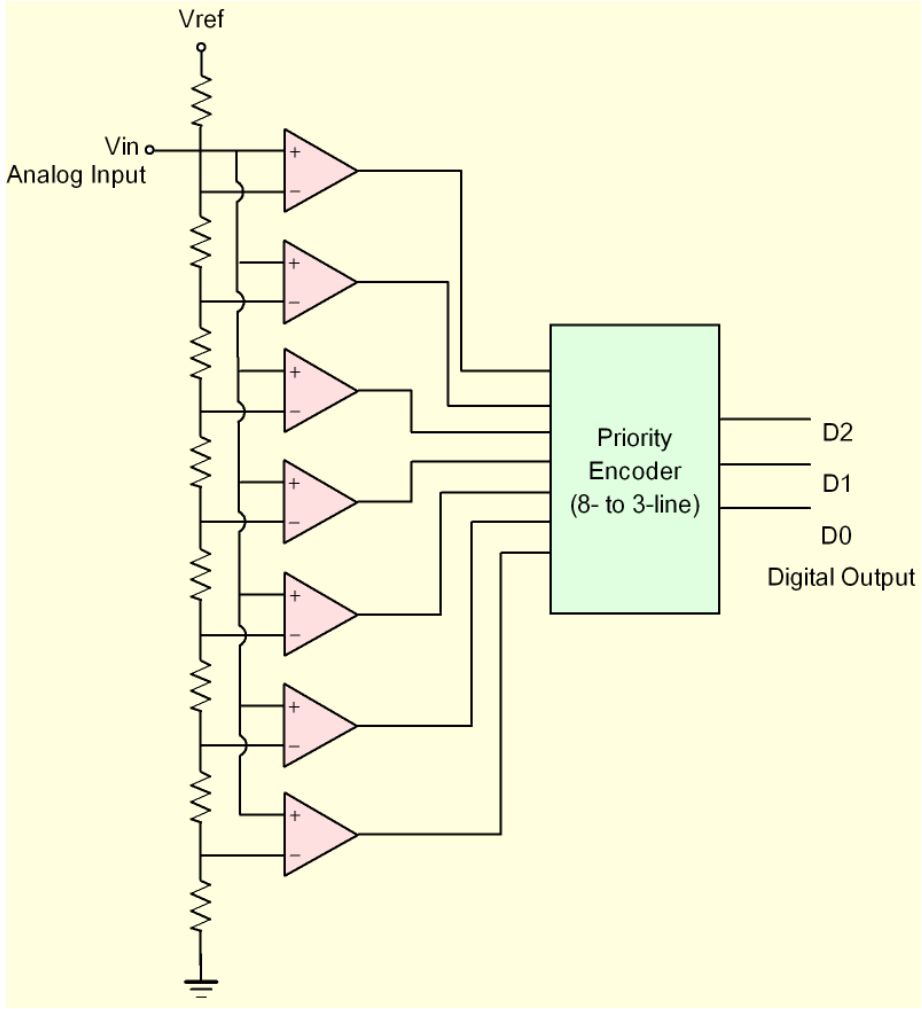
- Bit weight (LSB): the smallest voltage change
  - $\frac{V_{ref}}{2^n}$ Ex) 10bit ADC with Vref=5V:
  - LSB=5V/2^{10}=5/1025=4.88mV
- Full Scale (FS): the maximum digital output value of the ADC
  - $\frac{V_{ref}}{2^n}(2^n - 1) = V_{ref} - \text{LSB}$
  - Ex) (5/1024)*(1023)=5V-LSB = 4.995V
- Quantization Error: ADC rounds the analog signal
  - ±0.5 LSB For 5V, Quantization Error = ±2.44mV
- Sampling Theorem(Nyquist Theorem): The sample frequency should be at least twice the input frequency





ADC Types: Flash ADC

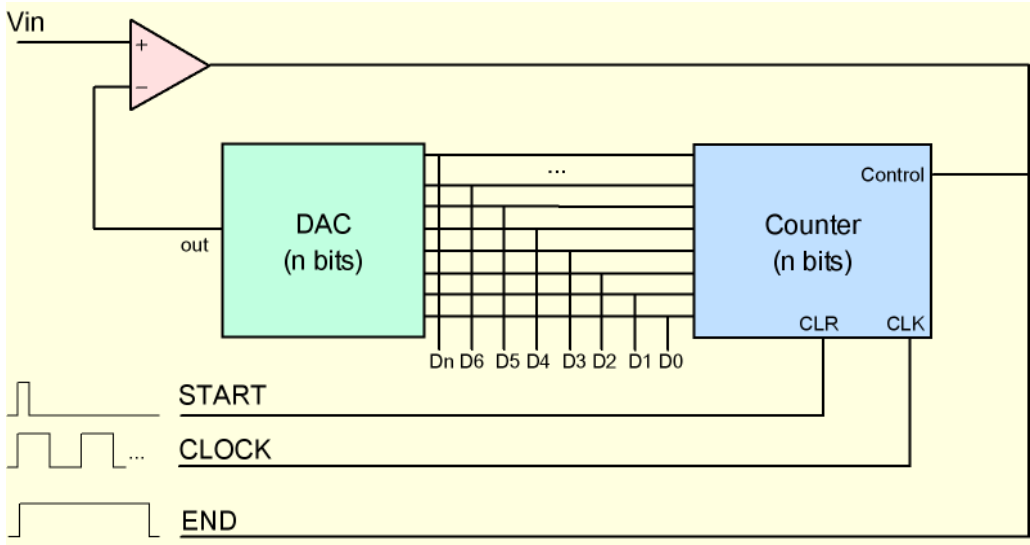
- Parallel Design
  - A resistor divider network generates discrete voltage levels
  - Input voltage is compared against all the voltage levels at once
  - Priority Encoder considers the first “HIGH” input from the top as valid and converts it to binary form.
- Advantage: Fast
  - Conversion takes just one cycle
- Disadvantage: A lot of components needed.
  - $2^n - 1$ comparators needed for n bit ADC



Picture Source: www.hardwaresecrets.com

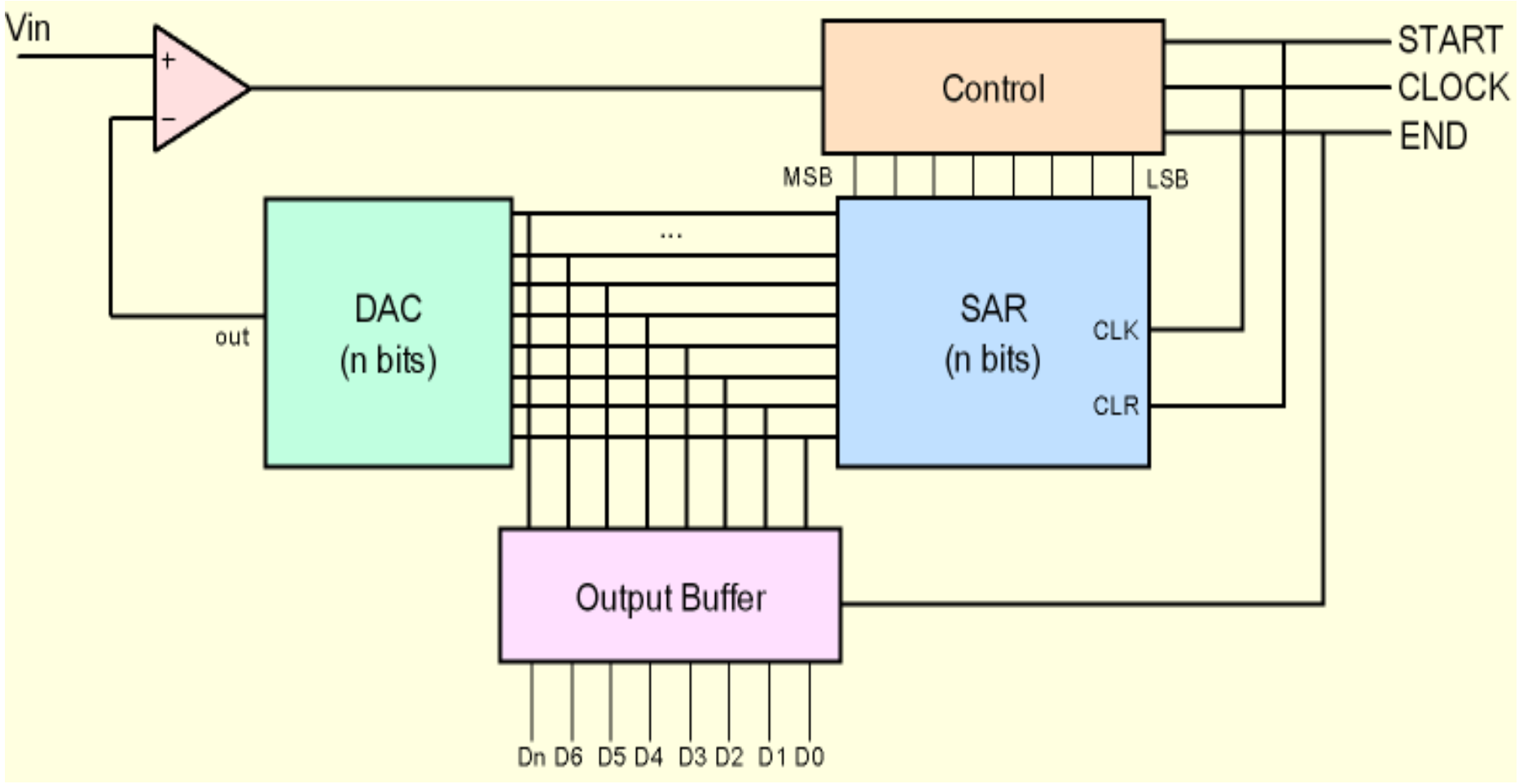
ADC Types: Ramp ADC

- Sequential Design
  - A Counter counts from $0 \cdots 2^n$
  - A DAC generates discrete voltage levels corresponding to the digital values $0 \cdots 2^n$ (i.e. a voltage Ramp)
  - In each cycle, input voltage is compared against the current voltage level generated by DAC
  - The comparator generates a "HIGH" value as soon as the ramp crosses the input value. The corresponding counter value becomes the output.
- Advantage: Only a few components needed.
- Disadvantage: Very slow.
  - $2^n - 1$ cycles (in worst case) for n bit ADC conversion



Picture Source: www.hardwaresecrets.com

ADC Types: Successive Approximation ADC



Picture Source: www.hardwaresecrets.com

ADC Types: Successive Approximation ADC

Step 1: Initialize Conversion

1.Start Conversion (SOC - Start of Conversion) signal is sent.

2.The SAR Register is set to half of the full scale (MSB = 1, others 0).

3.The DAC generates a voltage equal to half of Vref.

Step 2: Compare Analog Input with DAC Output

4.The Comparator compares the input voltage Vin with the DAC output:

- If Vin>VDACV, the bit remains 1.
- If Vin<VDACV, the bit is cleared (0).

5.The next bit is tested by setting it to 1.

Step 3: Repeat for Each Bit

6.This process is repeated bit-by-bit, from MSB to LSB.

7.After nnn iterations, the ADC completes and outputs the digital value.

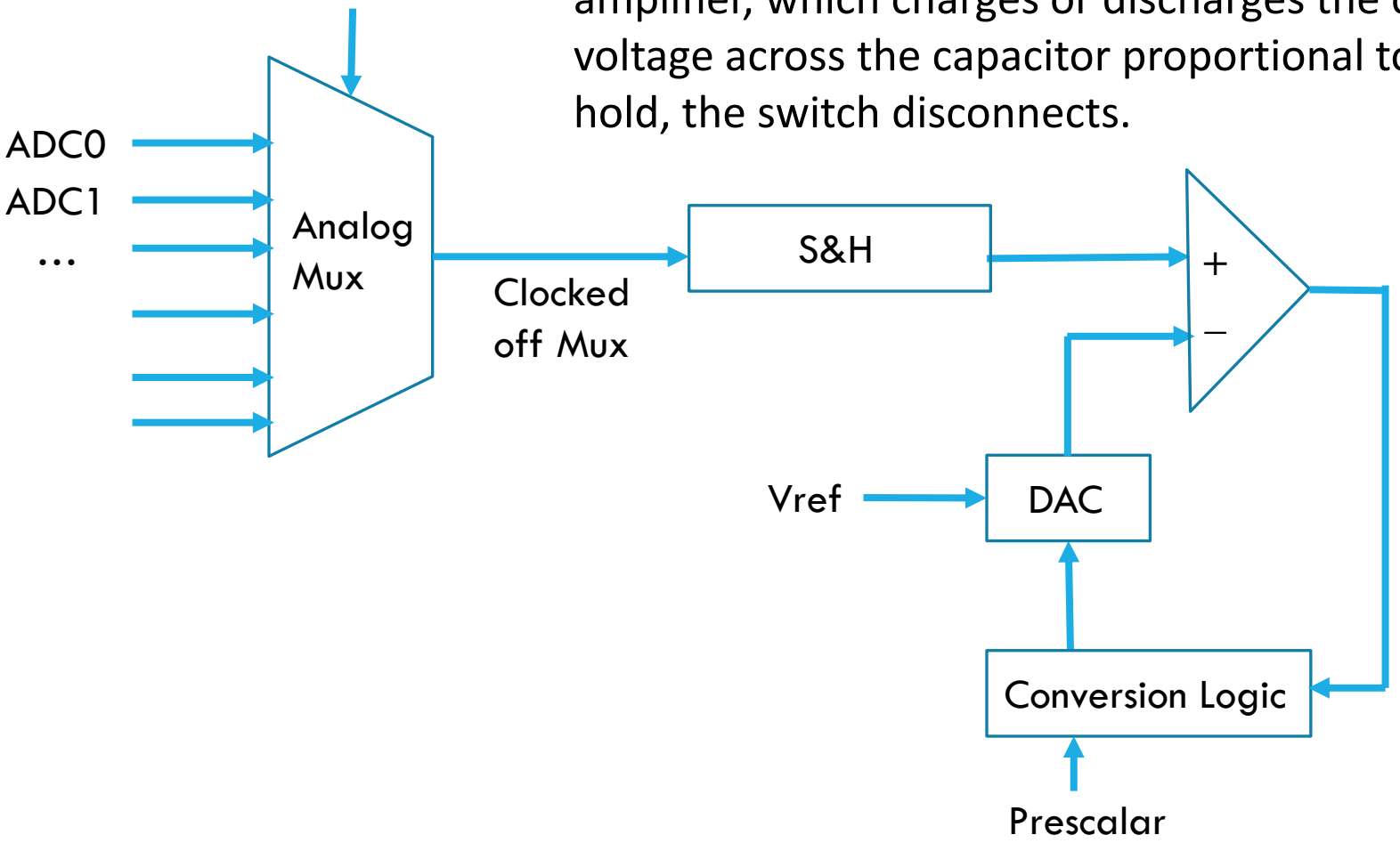
Step 4: Conversion Complete

8.The End of Conversion (EOC) signal is sent.

9.The final digital value is stored in the register.

ADC Types: Successive Approximation ADC

To sample, switch connects a capacitor to the output of a buffer amplifier, which charges or discharges the capacitor. This makes voltage across the capacitor proportional to the input voltage. To hold, the switch disconnects.



Conversion logic implements a successive approximation algorithm (a binary search; one bit per search):

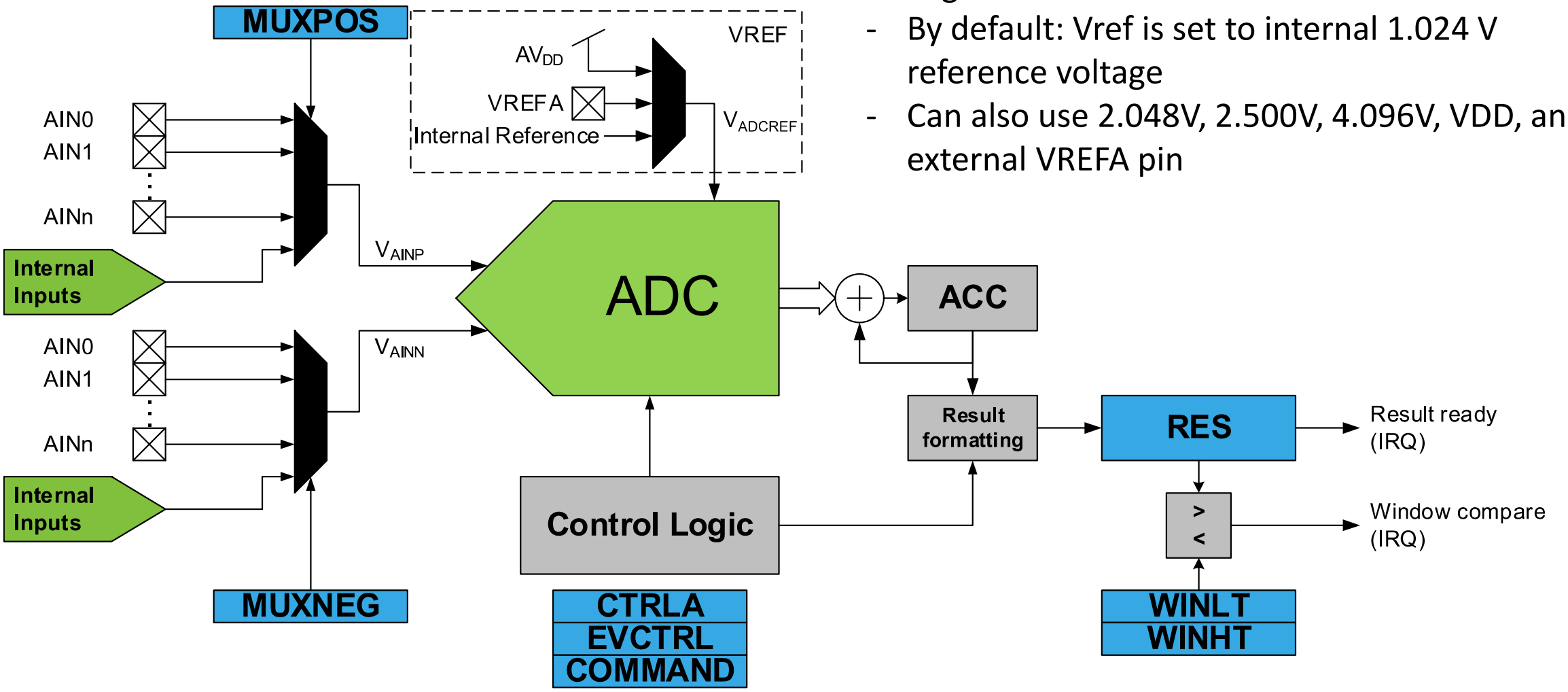
- DAC takes as input the output of the conversion logic and converts it to an analog voltage where Vref sets the full range
- Analog comparator decides whether the DAC output or input voltage is the largest

AVR128DB48 ADC

- ADC
  - 18 channels (most of the Digital I/O ports)
    - One channel at a time
  - 12-bit or 10-bit modes
  - Up to 130 Ks/s at 12-bit resolution
  - Different voltage references
  - Single mode and differential mode
  - Sample accumulation: average value

AVR128DB48 ADC Diagram

Figure 33-1. Block Diagram



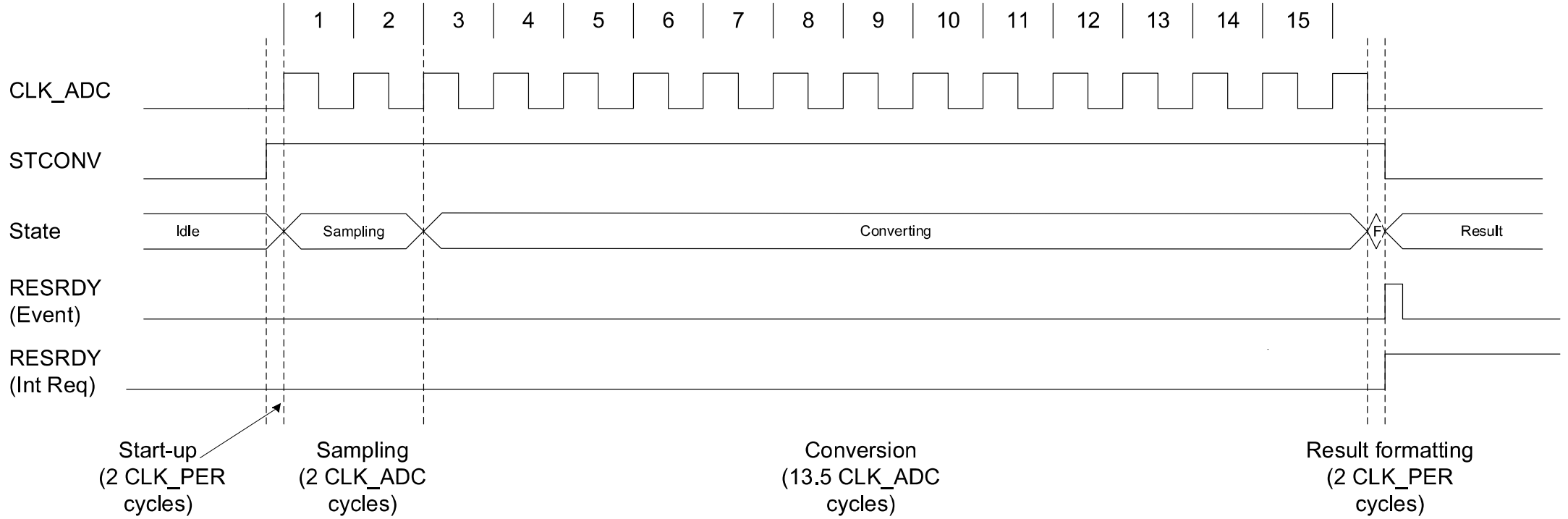
Voltage reference Vref:

- By default: Vref is set to internal 1.024 V reference voltage
- Can also use 2.048V, 2.500V, 4.096V, VDD, and external VREFA pin

Normal Conversion

- Total conversion time is $\frac{13.5+2}{fCLK\_ADC} + \frac{4}{fCLK\_PER}$

Figure 33-4. Timing Diagram - Single Conversion

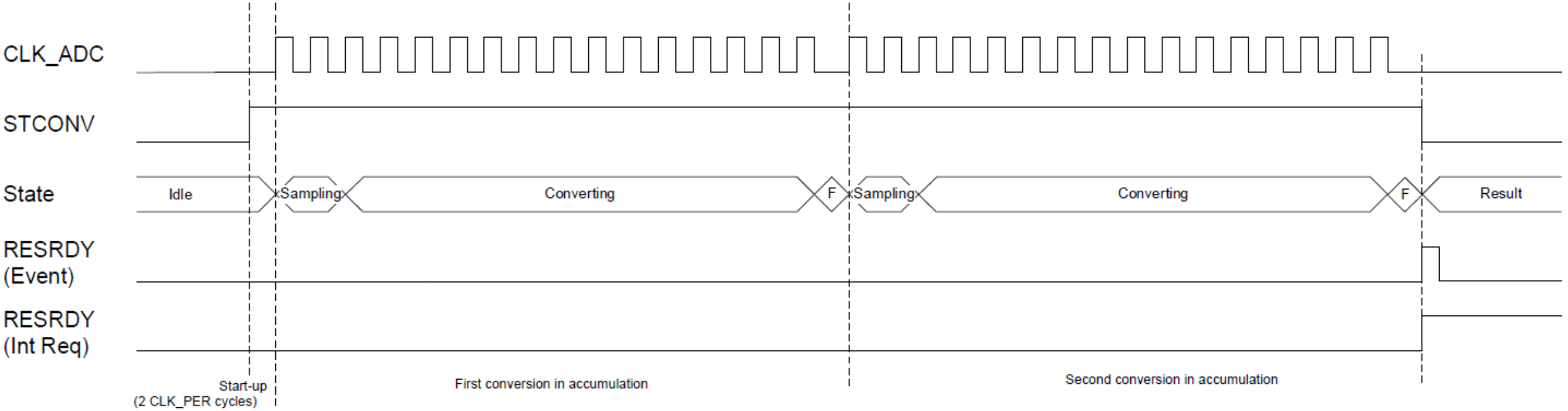


※ fCLK_ADC: ADC clock after prescaler, fCLK_PER: peripheral clock (before ADC prescaler)

Accumulated Conversion

$$\text{Total Conversion Time (12-bit)} = \frac{2}{f_{\text{CLK\_PER}}} + n\left(\frac{13.5 + 2}{f_{\text{CLK\_ADC}}} + \frac{2}{f_{\text{CLK\_PER}}}\right)$$

Figure 33-5. Timing Diagram - Accumulated Conversion



※ fCLK_ADC: ADC clock after prescaler, fCLK_PER: peripheral clock (before ADC prescaler)

Accuracy

- Noise: MCU produces up to 150mV line noise, there are other sources such as electrical field, etc.
  - Use capacitances close to the CPU to eliminate most of the inductance
- Capacitor in S&H leaks and can therefore not hold a value for too long
  - There exists a minimum sample speed/frequency
- Conversion logic takes time, so we cannot sample too fast
  - There exists a maximum sample speed/frequency
  - The faster you sample, the fewer number of accurate output bits (since the binary search cannot completely finish.)

Table 39-25. ADC Conversion Timing Specifications

Symbol	Description	Min.	Typ. †	Max.	Unit
T_{CLK_ADC} *	ADC clock period	0.5	—	8	µs

CTRLA: ADC Control Register A

Bit	7	6	5	4	3	2	1	0
	RUNSTDBY		CONVMODE	LEFTADJ	RESSEL[1:0]		FREERUN	ENABLE
Access	R/W		R/W	R/W	R/W	R/W	R/W	R/W
Reset	0		0	0	0	0	0	0

Bit 7 – RUNSTDBY Run in Standby

This bit determines whether the ADC still runs during Standby.

Value	Description
0	ADC will not run in Standby sleep mode. An ongoing conversion will finish before the ADC enters sleep mode.
1	ADC will run in Standby sleep mode

Bit 5 – CONVMODE Conversion Mode

This bit defines if the ADC is working in Single-Ended or Differential mode.

Value	Name	Description
0x0	SINGLEENDED	The ADC is operating in Single-Ended mode where only the positive input is used. The ADC result is presented as an unsigned value.
0x1	DIFF	The ADC is operating in Differential mode where both positive and negative inputs are used. The ADC result is presented as a signed value.

CTRLA: ADC Control Register A

Bit 4 – LEFTADJ Left Adjust Result

Writing a '1' to this bit will enable left adjustment of the ADC result.

Bits 3:2 – RESSEL[1:0] Resolution Selection

This bit field selects the ADC resolution. When changing the resolution from 12-bit to 10-bit, the conversion time is reduced from 13.5 CLK_ADC cycles to 11.5 CLK_ADC cycles.

Value	Description
0x00	12-bit resolution
0x01	10-bit resolution
Other	Reserved

Bit 1 – FREERUN Free-Running

Writing a '1' to this bit will enable the Free-Running mode for the ADC. The first conversion is started by writing a '1' to the Start Conversion (STCONV) bit in the Command (ADCn.COMMAND) register.

Bit 0 – ENABLE ADC Enable

Value	Description
0	ADC is disabled
1	ADC is enabled

CTRLB: ADC Control Register B

Bit	7	6	5	4	3	2	1	0
						SAMPNUM[2:0]		
Access						R/W	R/W	R/W
Reset						0	0	0

Bits 2:0 – SAMPNUM[2:0] Sample Accumulation Number Select

This bit field selects how many consecutive ADC sampling results are accumulated automatically. When this bit field is written to a value greater than `0x0`, the according number of consecutive ADC sampling results are accumulated into the ADC Result (ADCn.RES) register.

Value	Name	Description
`0x0`	NONE	No accumulation
`0x1`	ACC2	2 results accumulated
`0x2`	ACC4	4 results accumulated
`0x3`	ACC8	8 results accumulated
`0x4`	ACC16	16 results accumulated
`0x5`	ACC32	32 results accumulated
`0x6`	ACC64	64 results accumulated
`0x7`	ACC128	128 results accumulated

CTRLC: ADC Control Register C

33.5.3 Control C

Name: CTRLC
Offset: 0x02
Reset: 0x00

Bit	7	6	5	4	3	2	1	0
					PRESC[3:0]			
Access					R/W	R/W	R/W	R/W
Reset					0	0	0	0

Bits 3:0 – PRESC[3:0] Prescaler

This bit field defines the division factor from the peripheral clock (CLK_PER) to the ADC clock (CLK_ADC).

Value	Name	Description
0x0	DIV2	CLK_PER divided by 2
0x1	DIV4	CLK_PER divided by 4
0x2	DIV8	CLK_PER divided by 8
0x3	DIV12	CLK_PER divided by 12
0x4	DIV16	CLK_PER divided by 16
0x5	DIV20	CLK_PER divided by 20
0x6	DIV24	CLK_PER divided by 24
0x7	DIV28	CLK_PER divided by 28
0x8	DIV32	CLK_PER divided by 32
0x9	DIV48	CLK_PER divided by 48
0xA	DIV64	CLK_PER divided by 64
0xB	DIV96	CLK_PER divided by 96
0xC	DIV128	CLK_PER divided by 128
0xD	DIV256	CLK_PER divided by 256
Other	-	Reserved

- The CLK_ADC needs to be between 125 kHz and 2 MHz.
- A prescaler of 16 gives 16MHz/16 = 1 MHz
- To complete the binary search takes $\frac{13.5+2}{1\ MHz} + \frac{4}{16\ MHz}$ = 15.75 µs (63.5 ks/s)

CTRLD: ADC Control Register D

33.5.4 Control D

Name: CTRLD
Offset: 0x03
Reset: 0x00

Bit	7	6	5	4	3	2	1	0
	INITDLY[2:0]				SAMPDLY[3:0]			
Access	R/W	R/W	R/W		R/W	R/W	R/W	R/W
Reset	0	0	0		0	0	0	0

Bits 7:5 – INITDLY[2:0] Initialization Delay
This bit field defines the initialization delay before the first sample when enabling the ADC or changing to an internal reference voltage. Setting this delay will ensure that the components of the ADC are ready before starting the first conversion. The initialization delay will also be applied when waking up from a deep sleep to do a measurement. The delay is expressed as a number of CLK_ADC cycles.

Value	Name	Description
0x0	DLY0	Delay 0 CLK_ADC cycles
0x1	DLY16	Delay 16 CLK_ADC cycles
0x2	DLY32	Delay 32 CLK_ADC cycles
0x3	DLY64	Delay 64 CLK_ADC cycles
0x4	DLY128	Delay 128 CLK_ADC cycles
0x5	DLY256	Delay 256 CLK_ADC cycles
Other	-	Reserved

Bits 3:0 – SAMPDLY[3:0] Sampling Delay
This bit field defines the delay between consecutive ADC samples. This allows modifying the sampling frequency used during hardware accumulation, to suppress periodic noise that may otherwise disturb the sampling. The delay is expressed as CLK_ADC cycles and is given directly by the bit field setting.

Value	Name	Description
0x0	DLY0	Delay 0 CLK_ADC cycles
0x1	DLY1	Delay 1 CLK_ADC cycles
0x2	DLY2	Delay 2 CLK_ADC cycles
...	...	
0xF	DLY15	Delay 15 CLK_ADC cycles

Table 39-25. ADC Conversion Timing Specifications

Symbol	Description	Min.	Typ. †	Max.	Unit
t_{ADC_INIT} *	Initialization time	—	6	—	µs

MUXPOS Register

33.5.7 MUX Selection for Positive ADC Input

Bit	7	6	5	4	3	2	1	0
		MUXPOS[6:0]						
Access		R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset		0	0	0	0	0	0	0

Bits 6:0 – MUXPOS[6:0] MUX Selection for Positive ADC Input

This bit field selects which analog input is connected to the positive input of the ADC. If this bit field is changed during a conversion, the change will not take effect until the conversion is complete.

Value	Name	Description
0x00-0x15	AIN0-AIN21	ADC input pin 0-21
0x16-0x3F	-	Reserved
0x40	GND	Ground
0x41	-	Reserved
0x42	TEMPSENSE	Temperature sensor
0x43	-	Reserved
0x44	VDDDIV10	VDD divided by 10
0x45	VDDIO2DIV10	VDDIO2 divided by 10
0x46-0x47	-	Reserved
0x48	DAC0	DAC0
0x49	DACREF0	AC0 DAC Voltage Reference
0x4A	DACREF1	AC1 DAC Voltage Reference
0x4B	DACREF2	AC2 DAC Voltage Reference
Other	-	Reserved

MUXNEG Register

33.5.8 MUX Selection for Negative ADC Input

Bit	7	6	5	4	3	2	1	0
		MUXNEG[6:0]						
Access		R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset		0	0	0	0	0	0	0

Bits 6:0 – MUXNEG[6:0] MUX Selection for Negative ADC Input

This bit field selects which analog input is connected to the negative input of the ADC. If this bit field is changed during a conversion, the change will not take effect until the conversion is complete.

Value	Name	Description
0x00-0x0F	AIN0-AIN15	ADC input pin 0-15
0x10-0x3F	-	Reserved
0x40	GND	Ground
0x41-0x47	-	Reserved
0x48	DAC0	DAC0
Other	-	Reserved

RES: ADC Data Registers

33.5.15 Result

Bit	15	14	13	12	11	10	9	8
	RES[15:8]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
	RES[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	0	0

Bits 15:8 – RES[15:8] Result High Byte
This bit field constitutes the high byte of the ADCn.RES register, where the MSb is RES[15].

Bits 7:0 – RES[7:0] Result Low Byte
This bit field constitutes the low byte of the ADCn.RES register.

If LEFTADJ is set to 0,
- read RESL for low order bits, and until RESH is read, the ADC is locked out
- **read RES to get both RESH and RESL together**

LEFTADJ	RES[15:8]									RES[7:0]						
	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	0	0	0	0	Conversion [11:0]											
1	Conversion [11:0]												0	0	0	0

For 8-bit conversion, set LEFTADJ to 1 and read RESH

VREF.ADC0REF: ADC Voltage Reference

21.5.1 ADC0 Reference

Name: ADC0REF

Bit	7	6	5	4	3	2	1	0
	ALWAYSON					REFSEL[2:0]		
Access	R/W					R/W	R/W	R/W
Reset	0					0	0	0

Power consumption vs Conversion speed

Bit 7 – ALWAYSON Reference Always On

This bit controls whether the ADC0 reference is always on or not.

Value	Description
0	The reference is automatically enabled when needed
1	The reference is always on

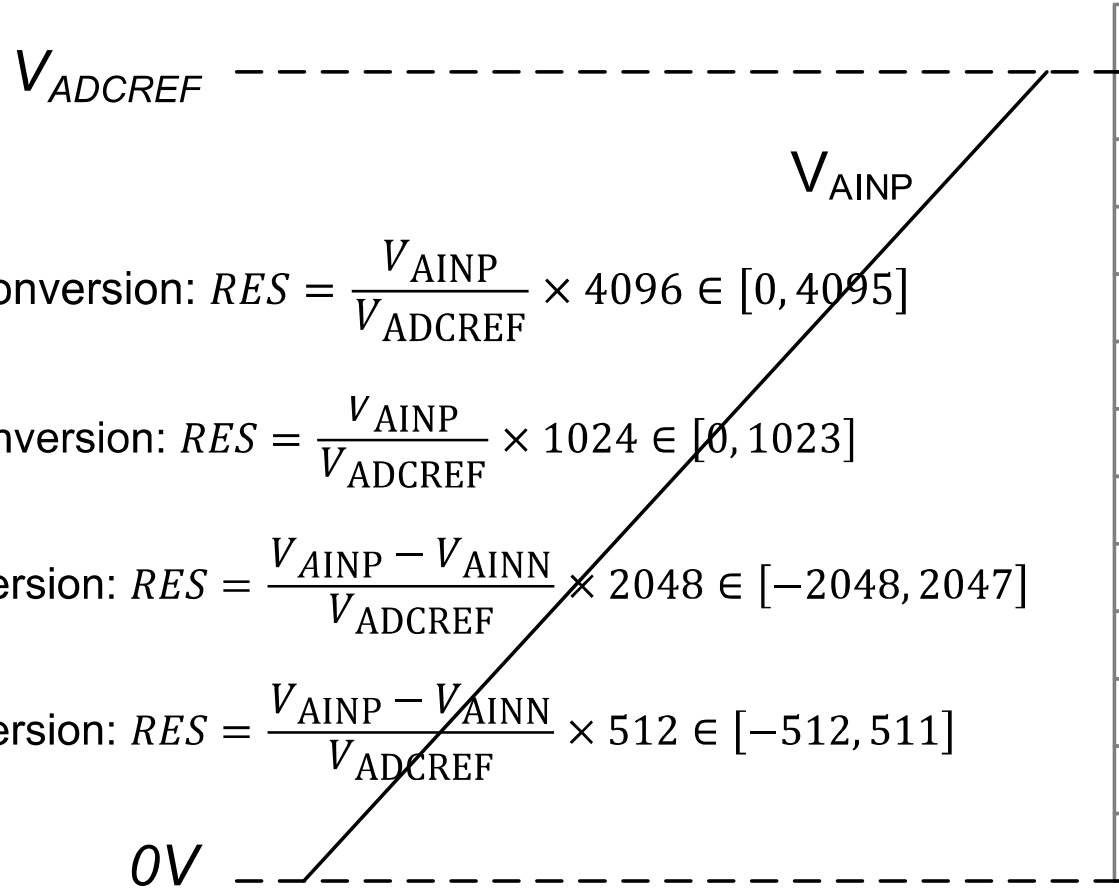
Bits 2:0 – REFSEL[2:0] Reference Select

This bit field controls the reference voltage level for ADC0.

Value	Name	Description
0x0	1V024	Internal 1.024V reference(1)
0x1	2V048	Internal 2.048V reference(1)
0x2	4V096	Internal 4.096V reference(1)
0x3	2V500	Internal 2.500V reference(1)
0x4	-	Reserved
0x5	VDD	VDD as reference
0x6	VREFA	External reference from the VREFA pin
0x7	-	Reserved

ADC Conversion

Figure 33-8. Unsigned Single-Ended, Input Range, and Result Representation



Dec	Hex	16-bit result
4095	FFF	0x0FFF
4094	FFE	0x0FFE
4093	FFD	0x0FFD
...	...	...
2049	801	0x0801
2048	800	0x0800
2047	7FF	0x07FF
2046	7FE	0x07FE
...	...	...
2	2	0x0002
1	1	0x0001
0	0	0x0000

Single-ended 12-bit conversion: $RES = \frac{V_{\mathrm{AINP}}}{V_{\mathrm{ADCREF}}} \times 4096 \in [0, 4095]$

Single-ended 10-bit conversion: $RES = \frac{V_{\mathrm{AINP}}}{V_{\mathrm{ADCREF}}} \times 1024 \in [0, 1023]$

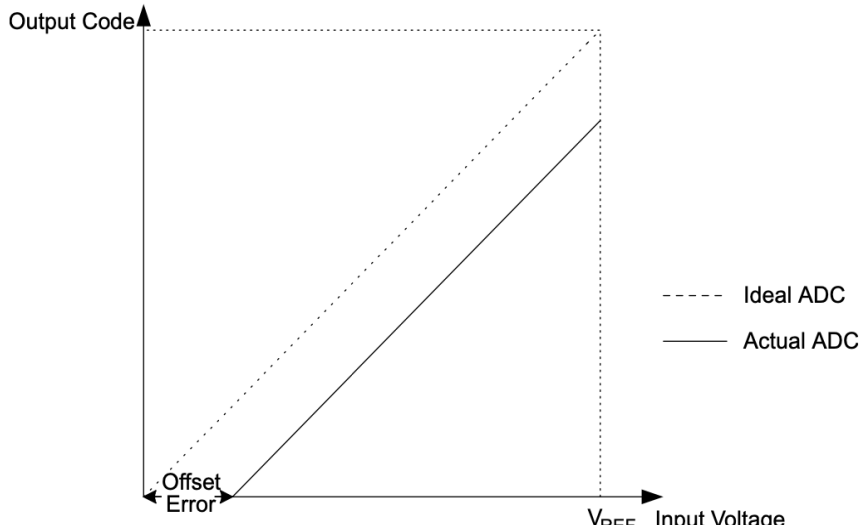
Differential 12-bit conversion: $RES = \frac{V_{AINP} - V_{\mathrm{AINN}}}{V_{\mathrm{ADCREF}}} \times 2048 \in [-2048, 2047]$

Differential 10-bit conversion: $RES = \frac{V_{\mathrm{AINP}} - V_{\mathrm{AINN}}}{V_{\mathrm{ADCREF}}} \times 512 \in [-512, 511]$

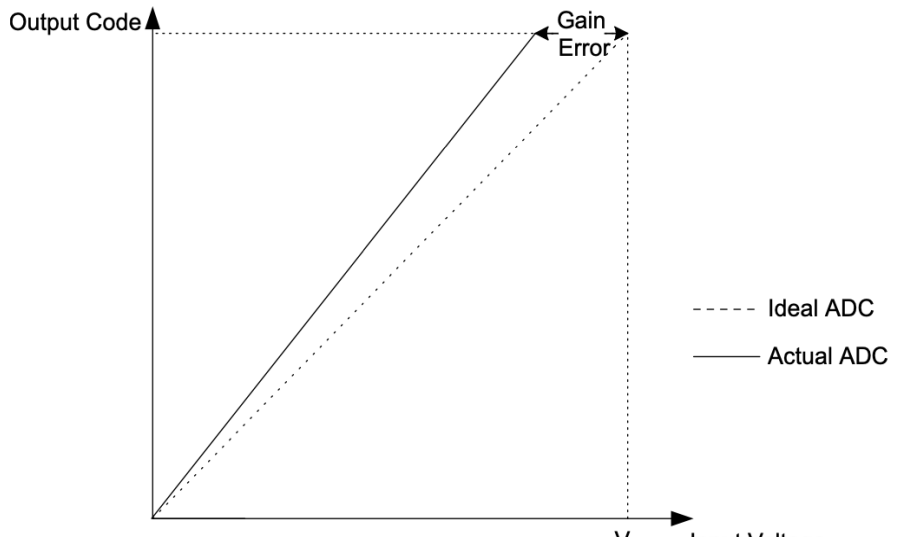
Where V_{AINP} is the single-ended or internal input.

ADC Conversion

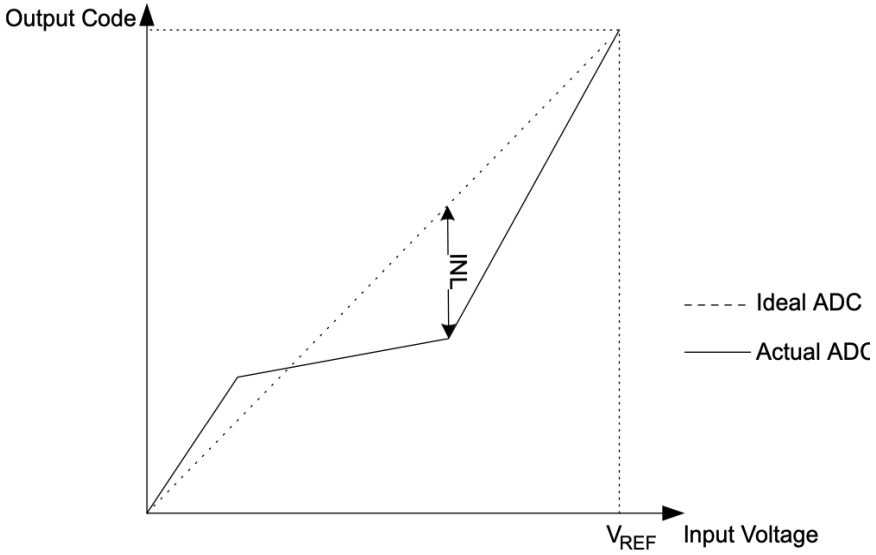
Offset Error



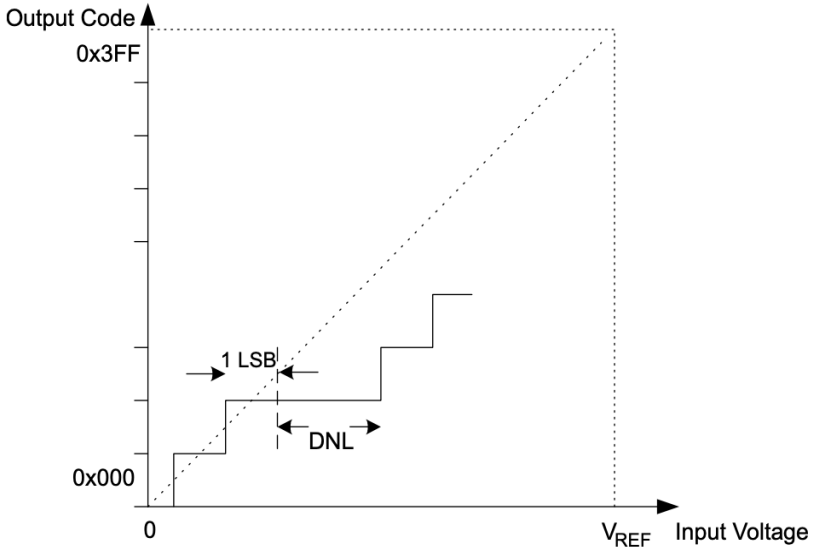
Gain Error



Integral Non-Linearity (INL)



Differential Non-Linearity (DNL)



ADC Conversion

Table 39-24. ADC Accuracy Specifications

Operating Conditions: • V_{ADCREF} = 3.0V • ADC in single ended conversion mode • f_{CLK_ADC} = 500 kHz						
Symbol	**Description**	**Min.**	**Typ. †**	**Max.**	**Unit**	**Conditions**
N_R	Resolution	—	—	12	bit	
E_{INL}	Integral nonlinearity error	-1.8	0.1	1.8	LSb	
E_{DNL}	Differential nonlinearity error(1)	-1	0.1	1	LSb	
E_{OFF}	Offset error	0	2.5	5	LSb	
E_{GAIN}	Gain error	-5	1.5	5	LSb	
V_{ADCREF} *	ADC reference voltage	1.024	—	V_{DD}	V	$f_{CLK\_ADC} \leq$ 500 kHz
		1.8	—	V_{DD}	V	

ADC Trigger Sources

Trigger Source	Use Case
Software Trigger	Manual control of ADC conversion.
Free-Running Mode	Continuous ADC sampling.
Timer Overflow	ADC starts periodically based on timer.
Event System	ADC triggered by GPIO change, USART RX, etc.
RTC Compare Match	ADC triggered at specific time intervals.

ADC Channel to Port Mapping

ADC Channel	PORT
AIN0	PDO
AIN1	PD1
AIN2	PD2
AIN3	PD3
AIN4	PD4
AIN5	PD5
AIN6	PD6
AIN7	PD7
AIN8	PE0
AIN9	PE1
AIN10	PE2

ADC Channel	PORT
AIN11	PE3
AIN12	
AIN13	
AIN14	
AIN15	
AIN16	PF0
AIN17	PF1
AIN18	PF2
AIN19	PF3
AIN20	PF4
AIN21	PF5

Not available on 48-pin package

Cannot be used as negative input for differential ADC

Example ADC code

```
int main(void)
{
        uart_init(3,9600,NULL);

        // input voltage at AIN6 (PORTD6)
        ADC0.MUXPOS = ADC_MUXPOS_AIN6_gc;

        // Set CLK_ADC to 1MHz (valid range from 125KHz to 2MHz)
        ADC0.CTRLC = ADC_PRESC_DIV16_gc;

        // Delay 16 CLK_ADC cycles - 16 us
        ADC0.CTRLD = ADC_INITDLY_DLY16_gc;

        // VDD as reference
        VREF.ADC0REF = VREF_REFSEL_VDD_gc;

        // enables the ADC circuitry and left adjusted
        ADC0.CTRLA = ADC_ENABLE_bm | ADC_LEFTADJ_bm;

        ADC0.COMMAND = ADC_STCONV_bm; // Start first A to D conversion
```

Conversion needs to finish

- Conversion needs to finish before the next conversion is called
  - Use a print statement
  - Delay functionality (of at least 16us)
  - Use interrupts
  - The most efficient solution is to poll

```
while ((ADC0.COMMAND & (1<< ADC_STCONV_bp)));
```

or

```
loop_until_bit_is_clear(ADC0.COMMAND, ADC_STCONV_bp);
```

STCONV will read as ‘1’ as long as a conversion is in progress.
When the conversion is complete, this bit is automatically cleared.

Example ADC code

```
loop_until_bit_is_clear(ADC0.COMMAND, ADC_STCONV_bp);

while (1)
{
        // Read from RESH to get the 8 MSB.  RESL needs to be read first
        int Ain = RESL; Ain = RESH;

        // Typecast integer into floating type data, divide by
        // maximum 8-bit value, and multiply by 3.3V for normalization
        float Voltage = (float)Ain/256.00 * 3.3;

        // Start next A to D conversion
        ADC0.COMMAND = ADC_STCONV_bm;

        // Write Voltage to string format and print
        // (minimum 4 char string with "." + 2 decimal places)
        char VoltageBuffer[6];
        dtostrf(Voltage, 4, 2, VoltageBuffer);
        printf("%s\n\r", VoltageBuffer);
}
```

print will take several ms – i.e. more than the ADC conversion time (15.75µs). So, no need to wait for STCONV bit to get cleared

Printing floats or doubles

- AVR libc library has a limited version of `printf`
  - `printf("%f", number)` doesn't work
  - Instead, do the following

```
char buffer[10];
dtostrf(number, 4, 2, buffer);
printf("%s", buffer);
```

The value should be of type double

The width is the minimum number of characters including the decimal point and possible negative sign

The precision is the number of digits after the decimal point

Lab Practice #10 Simple Voltmeter

Design a simple voltmeter that measures voltages between 0-3.3V with ~0.8mV resolution.

- Connect a potentiometer to produce variable voltage at AIN8 pin (PORTE0).
- Read the analog input voltage using ADC every 1s
  - Implement using tasks – no delay calls
- Convert the ADC reading to voltage measurement
- Print the voltage to the UART console.

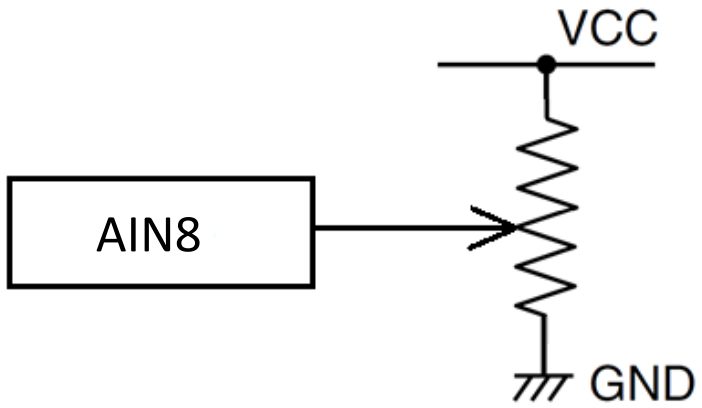
Note: Use the full 12-bit resolution of the ADC

Now you should be able to observe different voltage readings as you twist the potentiometer knob.

Hardware setting

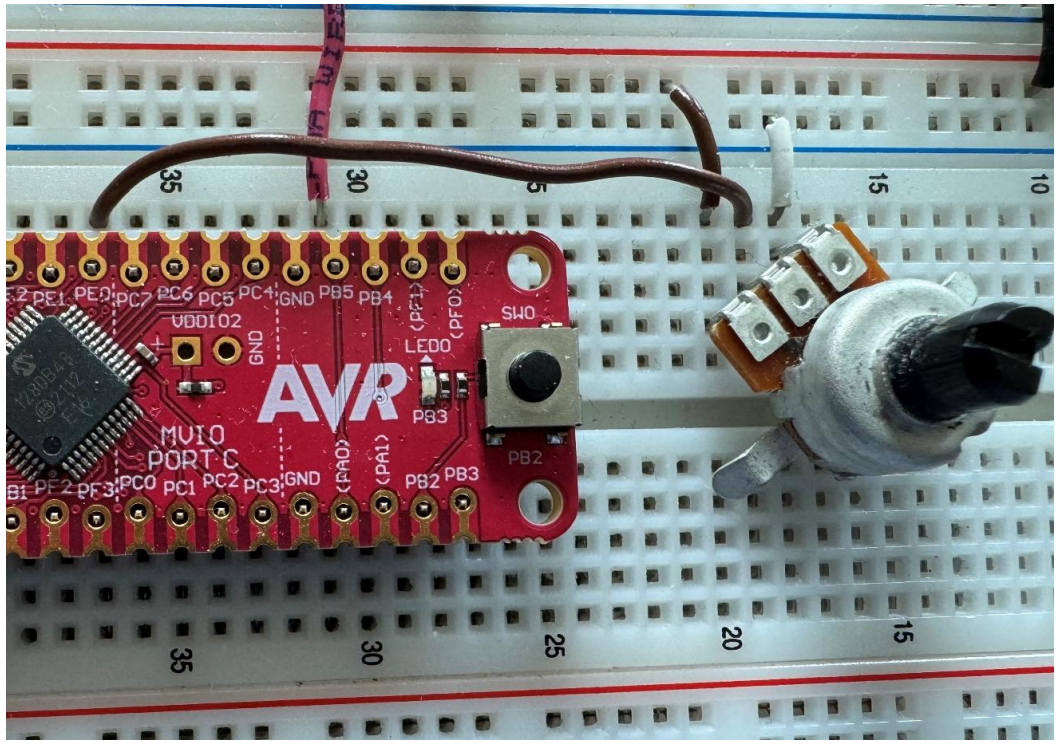
In this lab, we'll be measuring analog voltages using ADC

- Connect the potentiometer to AIN8 pin as shown below.



Connections

- Connect the board as shown in Fig.
- Use a potentiometer to get the sensing value to the ADC peripheral Pins AIN8 (PE0).



Connections for ADC

- Do not use delays. Use a counter to call the ADC task every second.
  - Use 1ms TCA0
- Set LEFTADJ to 0 (for 12-bit accuracy). Read data from RES not RESH
- When calculating voltage, convert to float before doing divide

Possible ADC related considerations

- Read multiple ADC channels
  - Select different ADC input channel
- Trigger ADC start of conversion by Timer
  - ADC Trigger options?
- Avoid blocking code during ADC conversion
  - ADC Result Ready Interrupt
- Sample accumulation
  - Higher accumulation improves noise filtering but takes longer